IMPLEMENTATION ISSUES

Implementation Pitfalls

- We learned about various cryptographic primitives and the provable security approach, and saw many secure constructions.
- You are almost ready to employ this knowledge in practice.
- Let us review some common mistakes one needs to be aware of and avoid when implementing cryptographic protocols.

Not Using the Right Primitives

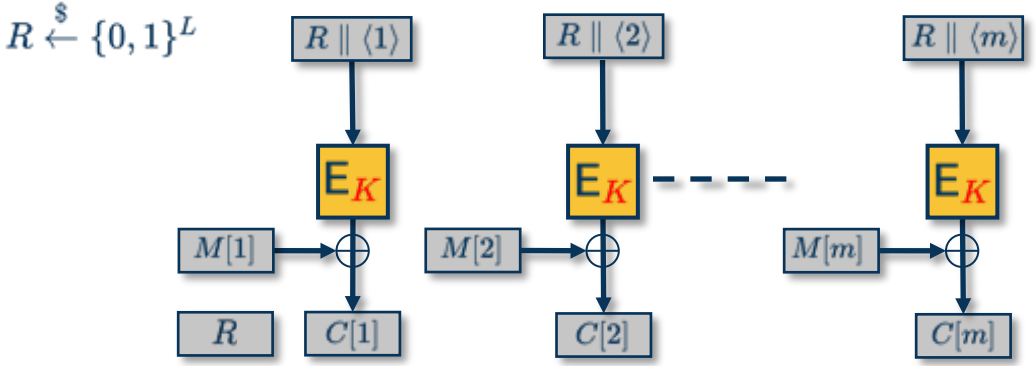
- ATM-based passive optical networks commonly use a block cipher called CHURN. Its key size is 8 bits and its block size is 4 bits!

Using the Constructs without Security Proofs

- The use of the ECB mode and the Plain RSA encryption is still very common.

Not Considering the Security Bounds

Consider the encryption algorithm of a scheme $\mathsf{CTRS\$}[L]$
Let $\mathsf{E} : \{0,1\}^k \times \{0,1\}^n \to \{0,1\}^n$ be a block cipher.



One can prove that for every $\mathcal{A}$ making q queries there exists $\mathcal{B}$ s.t.

$$\mathbf{Adv}^{\text{ind-cpa}}_{\mathsf{CTRS\$}[L]}(\mathcal{A}) \leq \mathbf{Adv}^{\text{prf}}_{\mathsf{E}}(\mathcal{B}) + \frac{q^2}{2^{L+1}}$$

Is CTRS$ secure?

Wired Equivalent Privacy (WEP) is a for IEEE 802.11 wireless networks

$$\mathbf{Adv}^{\text{ind-cpa}}_{\text{CTRS\$}[L]}(\mathcal{A}) \leq \mathbf{Adv}^{\text{prf}}_{\text{E}}(\mathcal{B}) + \frac{q^2}{2^{L+1}}$$

- WEP protocol for IEEE 802.11 wireless networks uses a scheme like CTRS$ with L = 24, 40, 64 or 80.
- Assume L = 24 and q = 4096. Then the last term becomes 1/2 and no security is guaranteed by the bound!

Not Using the Right Tool

- It is tempting to believe that encryption provide some authenticity.
- The first versions of the SSH protocol, IPsec specification and the WEP protocol did not use message authentication codes, and thus were subject to certain attacks.

Not Implementing Exactly the Provable-Secure Schemes

- Diebold voting machines encrypted the votes with CBC$, but used all-zero string as an IV.
- Microsoft Word and Excel used CBC$, but did not pick a new random IV each time.
- In many protocols, the IV used when encrypting a message is the last block of the previous ciphertext.

Random Numbers

- It is usually straightforward to implement the pseudo-code descriptions in C or Java.
- However, how do you implement commands like $K \xleftarrow{\$} \{0,1\}^k$?
- The C offers a built-in random number generator, that works roughly as this

32-bit number

Procedure srand $(seed)$

$state = seed;$

Function rand ()

$state = ((state * 1103515245) + 12345) \bmod 2147483648;$

return $state$

2^{31}

- So one can implement $K \xleftarrow{\$} \{0,1\}^k$ as follows

Algorithm $\mathcal{K}$

$K \xleftarrow{\$} \{0,1\}^{128}$

return K

Function keygen()

$key[0] = rand();\ key[1] = rand();$

$key[2] = rand();\ key[3] = rand();$

return key

- But looking at how rand() works we notice that

$$
\begin{aligned}
key[1] =& ((key[0] \cdot 1103515245) + 12345) \mod 2^{31} \\
key[2] =& ((((key[0] \cdot 1103515245) + 12345) \cdot 1103515245) + \\
& 12345) \mod 2^{31} \\
key[3] =& ((((((key[0] \cdot 1103515245) + 12345) \cdot 1103515245) + \\
& 12345) \cdot 1103515245 + 12345) \mod 2^{31}
\end{aligned}
$$

- This means that there are still only 2^{32} possibilities for the key

Randomness for Encryption

- Designers of SSH, IPsec, TLS all assumed that the last blocks of the ciphertexts in CBC can be used as IVs for the next ciphertexts.
    - Lead to BEAST attack on TLS 1.0

Combining the Schemes

- Recall that it is insecure in general to apply the Encrypt-and-MAC paradigm in order to achieve both privacy and authenticity.

Key Management

- All users of the WEP encryption protocol use the same symmetric key.
- The key for the secure votes encryption in Diebold machines is hardwired in the code:

```
#define DESKEY ((des_key*)”F2654hD4”)
```

Provably Secure Schemes Can be Attacked Too

- Even if implemented correctly, provably secure schemes can be attacked too
- The main reason is the formal security definition may not capture some attacks
  - Lucky 13 attack on TLS
  - Plaintext recovery attack on SSH
  - The cryptographic schemes in both protocols have been proven secure, but the security definitions did not take into account that an attacker can
    - Distinguish between decryption failures of different types (e.g. MAC failure and wrong padding/message length)
    - Ciphertexts can be fragmented (the receiver gets a stream and doesn't know where are the borders between individual ciphertexts)

Summary

- Use widely accepted and believed to be secure building blocks (e.g. AES).
- Use provably secure (under reasonable assumptions) constructions (e.g. CBC$).
- Do not assume that the schemes provide security properties other than what is proven about them (e.g. encryption does not provide authenticity).
- Realize that the use of a provably secure scheme does not guarantee that the entire system will be secure.
- Make sure that you implement exactly the scheme that was proven secure.

Reference

- Yoshi Kohno “Implementation pitfalls”. Available at https://www.cc.gatech.edu/~aboldyre/kohno.pdf